Lecture 8 - Input/Output (I/O) Organization

1. Purpose of I/O Modules

The CPU and main memory are the core components of a computer system, but they cannot function in isolation. They need to interact with the outside world – with users, storage devices, and networks. This interaction is facilitated by **Input/Output (I/O) modules**.

The primary purposes of I/O modules are:

- **Bridging the Speed Gap:** CPUs operate at extremely high speeds, while I/O devices (like hard drives or printers) are significantly slower. I/O modules act as a buffer and speed matcher, allowing the CPU to continue processing while I/O operations are underway.
- **Handling Device Peculiarities:** Each I/O device has its own unique operational characteristics, data formats, and control requirements. I/O modules translate generic I/O commands from the CPU into specific commands understood by the device, and vice versa. They manage device-specific registers and control signals.
- **Buffering and Synchronization:** I/O modules often contain internal buffers to temporarily hold data during transfers between the CPU/memory and the I/O device. They also synchronize the flow of data to match the speeds of both sides.
- **Error Detection:** Many I/O modules include error detection capabilities to ensure data integrity during I/O operations.
- **Address Translation (for some devices):** In systems with virtual memory, I/O modules may assist in translating virtual addresses to physical addresses for direct memory access (DMA).
- **Resource Sharing:** In multi-user or multi-tasking environments, I/O modules help manage access to shared I/O devices by multiple processes.

In essence, an I/O module offloads much of the I/O processing burden from the CPU, making the system more efficient and flexible.

2. I/O Techniques

There are three primary techniques used for managing I/O operations, each with its own advantages and disadvantages regarding CPU involvement and efficiency.

2.1 Programmed I/O (PIO)

- **Concept:** In Programmed I/O, the CPU directly controls the I/O operation from start to finish. The CPU issues an I/O command to an I/O module and then continuously monitors the status of the I/O device until the operation is complete.
- **Process:**
    1. The CPU writes a command to the I/O module's command register.
    2. The CPU reads the I/O module's status register repeatedly (polling) to check if the device is ready or if the operation has completed.
    3. If reading data, the CPU reads data from the I/O module's data register.
    4. If writing data, the CPU writes data to the I/O module's data register.
- **CPU Involvement:** High. The CPU is fully occupied during the I/O operation and cannot perform other tasks. It wastes significant cycles waiting for slow I/O devices.
- **Advantages:** Simple to implement from a hardware perspective.
- **Disadvantages:** Inefficient for slow I/O devices; the CPU idles unnecessarily. Not suitable for multitasking environments.
- **Use Cases:** Very simple, low-speed devices, or in embedded systems where CPU time might not be a bottleneck for specific tasks.

2.2 Interrupt-Driven I/O

- **Concept:** To overcome the inefficiency of Programmed I/O, the CPU is freed from constantly polling the I/O device. Instead, the I/O module generates an interrupt signal to the CPU when an I/O operation is complete or when the device requires attention.
- **Process:**
    1. The CPU issues an I/O command to the I/O module.
    2. The CPU then continues with other tasks (e.g., executing another program).
    3. When the I/O operation is complete (or data is ready), the I/O module generates an interrupt request to the CPU.
    4. The CPU receives the interrupt, saves its current context, and jumps to an **Interrupt Service Routine (ISR)**.
    5. The ISR handles the I/O completion (e.g., transferring data, acknowledging the interrupt).
    6. After the ISR finishes, the CPU restores its context and resumes the task it was performing before the interrupt.

- **CPU Involvement:** Moderate. The CPU is interrupted only when necessary. It can perform other tasks while I/O is in progress, but data transfer between the I/O module and memory/CPU still involves the CPU.
- **Advantages:** More efficient than Programmed I/O, especially for devices with varying or unpredictable completion times. Allows for multitasking.
- **Disadvantages:** Each data transfer (e.g., byte by byte or word by word) still requires a CPU intervention (an interrupt and ISR execution), which can be inefficient for high-volume data transfers.
- **Use Cases:** Keyboards, mice, low-speed modems, and other devices where data transfers are not continuous and high-volume.

2.3 Direct Memory Access (DMA)

- **Concept:** DMA is the most efficient I/O technique for high-speed, high-volume data transfers. It allows I/O modules to transfer data directly to and from main memory without involving the CPU for each word or byte. This requires a dedicated hardware component called a **DMA Controller (DMAC)**.
- **Process:**
    1. The CPU programs the DMA Controller with:
        - The starting address in main memory for the transfer.
        - The starting address of the I/O device's buffer.
        - The number of bytes/words to transfer.
        - The direction of transfer (read from device to memory, or write from memory to device).
    2. The CPU then issues a command to the I/O module to initiate the DMA transfer and proceeds with other tasks.
    3. The DMA Controller takes over, independently managing the data transfer between the I/O device and main memory. It uses the system buses directly.
    4. Once the entire block transfer is complete, the DMA Controller generates a single interrupt to the CPU.
    5. The CPU's ISR then handles the completion of the entire block transfer.
- **CPU Involvement:** Low. The CPU is only involved at the beginning (setting up the DMA) and at the end (receiving the completion interrupt). It is largely free during the actual data transfer.
- **Advantages:** Extremely efficient for large data transfers. Frees the CPU for other computations, significantly improving system throughput. Essential for high-speed devices.

- **Disadvantages:** Requires more complex hardware (DMA controller).
- **Use Cases:** Disk drives (HDDs, SSDs), network interfaces (NICs), graphics cards, and other high-bandwidth I/O devices.

3. I/O Interfaces (Ports, Controllers)

To facilitate communication between the CPU/memory and I/O devices, dedicated interfaces are used.

- **I/O Ports:**
  - **Concept:** These are essentially memory-mapped or isolated I/O addresses that the CPU uses to communicate with I/O modules and devices.
  - **Registers:** Each I/O device or controller typically exposes a set of registers that the CPU can read from or write to:
    - **Control Register:** CPU writes commands to this register (e.g., start transfer, reset device).
    - **Status Register:** CPU reads this to check the device's current state (e.g., busy, ready, error).
    - **Data Register:** Used for actual data transfer (either CPU reads from it or writes to it).
  - **Addressing:**
    - **Memory-Mapped I/O:** Device registers are treated as if they are locations in main memory. The CPU uses its regular memory read/write instructions to access them. This is simpler to implement but reserves a portion of the memory address space for I/O.
    - **Isolated I/O:** Devices have their own separate address space, accessed using special CPU I/O instructions (e.g., IN and OUT in x86 architectures). This separates I/O addresses from memory addresses but requires dedicated instructions.
- **I/O Controllers (or Device Controllers):**
  - **Concept:** These are the hardware components that implement the logic for an I/O module. They manage the specific electrical and logical requirements of a particular type of I/O device.
  - **Functionality:**
    - Convert serial data from the device to parallel data for the bus, and vice versa.

        - Manage device-specific protocols and timing.
        - Provide buffering for data.
        - Implement error detection and correction where applicable.
        - Handle interrupts and DMA requests.
    - **Examples:** Disk controllers (SATA, NVMe), USB controllers, Ethernet controllers, graphics card controllers.

4. Introduction to Common I/O Devices

Understanding the I/O techniques is enhanced by knowing the devices they control.

- **Keyboard:**
    - **Type:** Input device.
    - **I/O Technique:** Primarily **Interrupt-Driven I/O**. Each key press generates a scan code that is typically buffered by the keyboard controller. The controller then interrupts the CPU to signal that data is available. Very low data rate.
- **Mouse:**
    - **Type:** Input device.
    - **I/O Technique:** Primarily **Interrupt-Driven I/O**. Movement or clicks generate data packets that are sent to the controller, which then interrupts the CPU. Low data rate.
- **Display (Monitor/Screen):**
    - **Type:** Output device.
    - **I/O Technique:**
        - Early displays used **Programmed I/O** or **Interrupt-Driven I/O** for simple character displays.
        - Modern graphical displays extensively use **DMA**. The CPU (or GPU) prepares the frame buffer (the image data) in main memory. The display controller (often part of the GPU) then uses DMA to continuously read this frame buffer and send it to the display at refresh rates (e.g., 60 times per second). High data rate.
- **Disk Drives (HDD, SSD):**
    - **Type:** Storage (Input/Output) device.
    - **I/O Technique:** Primarily **DMA**. Disk transfers involve large blocks of data (sectors, pages/blocks). DMA is essential to efficiently move data between the

disk controller's buffer and main memory without constantly tying up the CPU. High data rate.

Activities: Comparing Different I/O Techniques

Let's compare the I/O techniques based on several criteria:

Feature	**Programmed I/O (PIO)**	**Interrupt-Driven I/O**	**Direct Memory Access (DMA)**
CPU Involvement	High (CPU polls continuously)	Moderate (CPU interrupted for each transfer/event)	Low (CPU sets up, then receives single interrupt for block)
Efficiency	Very Low (CPU wasted cycles)	Medium (Better than PIO, but still CPU overhead)	High (CPU largely free during transfer)
Data Transfer Unit	Typically byte or word	Typically byte or word	Block of data (multiple bytes/words)
Hardware Complexity	Simple	Moderate (requires interrupt controller)	High (requires DMA controller)
Latency	Can be low for *CPU-initiated* transfers, but overall slow process	Responsive to events, but overhead per transfer adds latency	Low for bulk transfers once set up
Throughput	Very low	Medium	Very high
Best Use Case	Very slow, simple devices; microcontrollers	Keyboards, mice, serial ports, low-speed networks	Disk drives, network cards, graphics cards, high-speed peripherals